Authorisation

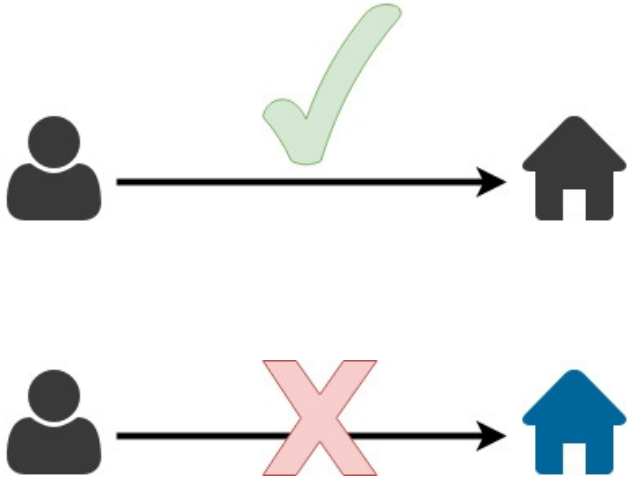
Secure Programming

Michael Lehmann

Basics

Authorisation

- Managing permissions of **subjects accessing objects**
- permission = access(subject, object)
- Access can be
  - **C**reate
  - **R**ead
  - **U**pdate
  - **D**elete
  - Execute
  - None
- Indicates the **actions** the subject may perform on the object



Different methods and policies exist…

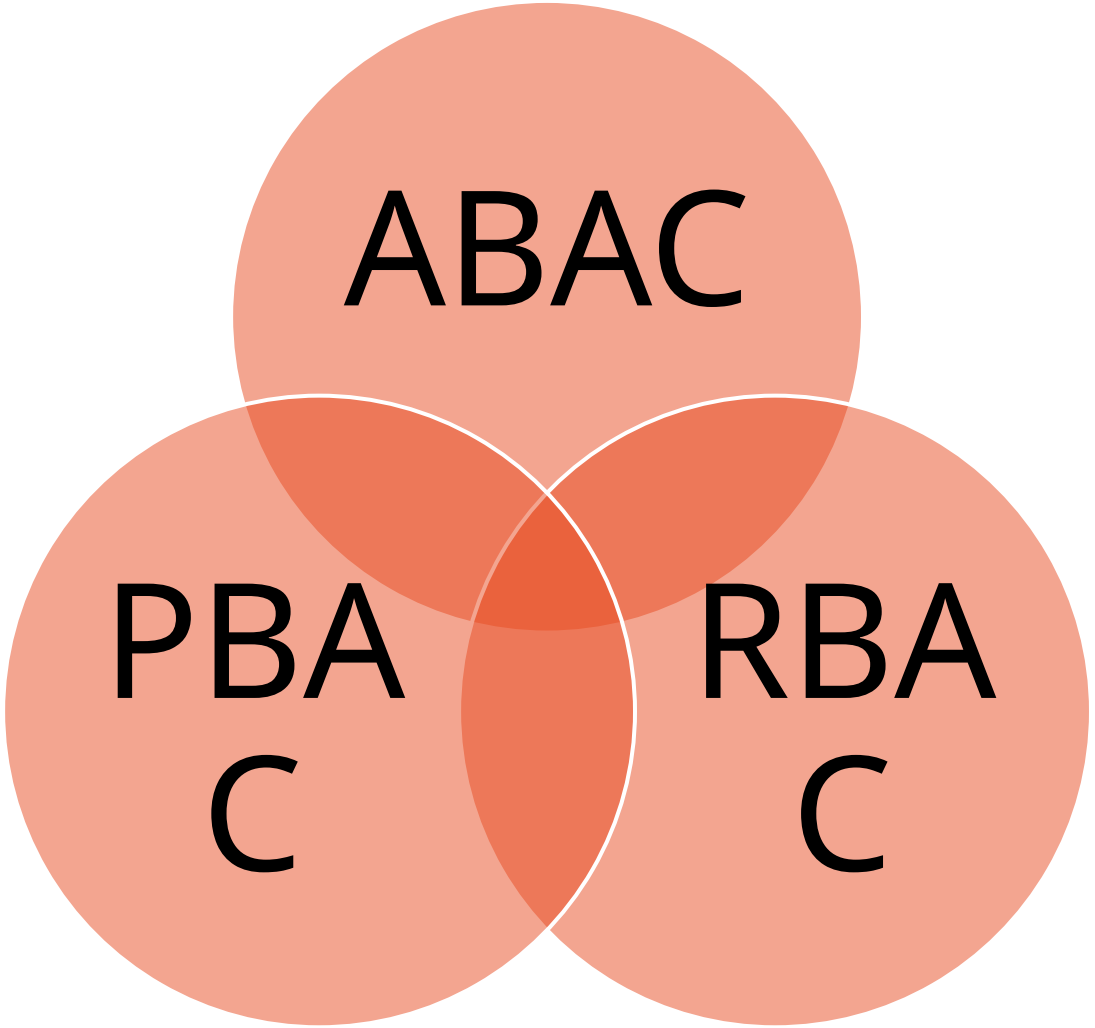
Methods

- Discretionary Access Control
- Mandatory Access Control

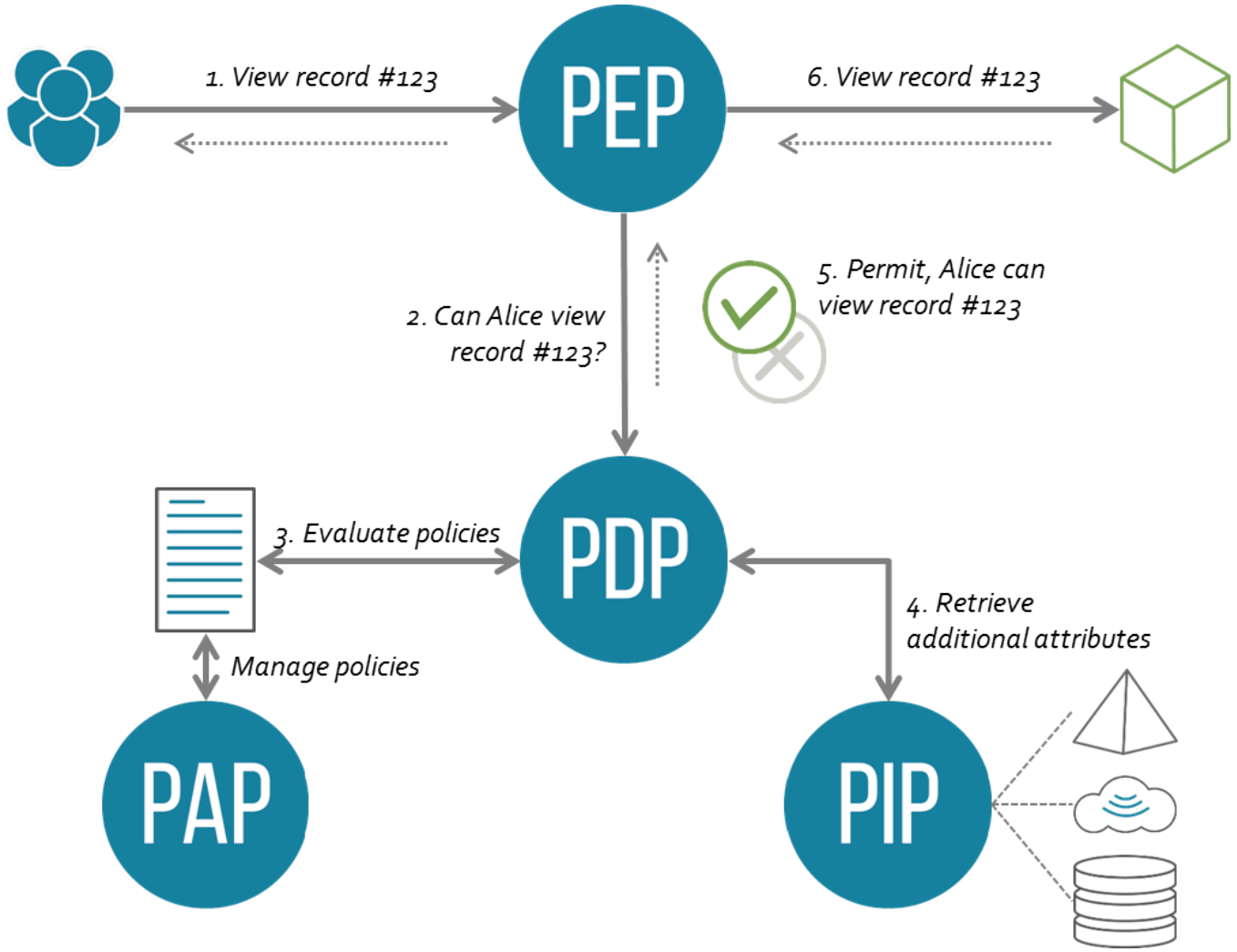
Policies

- Open Policy
- Closed Policy

…and also different models



XACML Architecture Model



Source: Wikipedia

An API is usually structured in one of two ways

Resource-oriented architecture

- Business logic: mostly frontend
- API structure: along objects
- Object-level based access control

Service-oriented architecture

- Business logic: mostly backend
- API structure: along services
- Function-level based access control

In resource based APIs, http methods determine the action

Action	HTTP Method	URI	Request Body	Response Body
Read	GET	Specific: /object/5 List all: /object/	-	{ "id": 5, "name": "someobject", "country": "Switzerland", "city": "Luzern" }
Create	POST / PUT	/object/6	{ "name": "myobject", "country": "Switzerland", "city": "Bern" }	{"id": 6, "name": "myobject", "country": "Switzerland", "city": "Bern" }
Update	PATCH / PUT	/object/6	{"id": 6, "name": "myobject", "country": "Switzerland", "city": "Zurich" }	{"id": 6, "name": "myobject", "country": "Switzerland", "city": "Zurich" }
Delete	DELETE	/object/6	-	-

Role-Based Access Control Matrix

- **Access control matrix** for a resource-oriented online banking application
- Define access per role and resource

Role	Account /account/	News /news/	Transactions /transaction/
User			
PR-Employee			
Client Advisor			
Public			

1. Fill in access action (Create, Read, Update, Delete)
2. Who can see which account?

Issues

Potential issues are manyfold

- Misconfigurations
- Insecure IDs
- Forced browsing past access control checks
- Path traversal
- LFI/RFI
- Client-side caching
- Access control based on untrusted input
- Missing function level access control

Broken Access Control

- Insecure direct object reference
  - An application provides direct access to objects based on user-supplied input: https://example-bank.com/account/details?accountId=1234
  - Entitlements of current user not checked

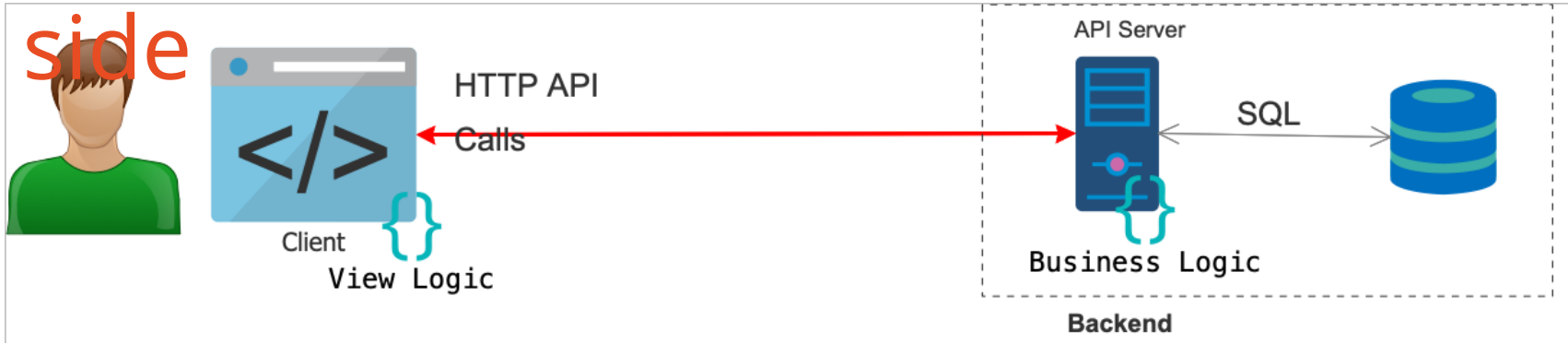
```
String query = "SELECT * FROM accts WHERE account = ?";

PreparedStatement pstmt = connection.prepareStatement(query , … );

pstmt.setString( 1, request.getParameter("acct"));

ResultSet results = pstmt.executeQuery( );
```

- Path traversal
  - Tricks URI pattern-based access controls rules
    - MANAGER can access the resources with URI /managers/**
    - USER can only access /users/**

    → but /users/../managers/salaries.xhtml returns the page as well

Source: https://www.owasp.org/

Access control must be done on server side



Displayed on Client:

First Name	Last Name	Roles
Peter	Muster	BACK_OFFICE_ADMIN
Andre	Hess	PRODUCT_ENG

JSON Data sent to client:

[
{employeeId:2378,firstName:"Peter",lastName:"Muster",
roles:["EMPLOYEE","BACK_OFFICE_ADMIN"],**salary:50000**},
{employeeId:3485,firstName:"Andre",lastName:"Hess",
roles:["EMPLOYEE","PRODUCT_ENG"],**salary:60000**},
{employeeId:1024,firstName:"Elon",lastName:"Musk",
roles:["OWNER","BOARD_MEMBER"],**salary:5000000**},
{employeeId:1301,firstName:"Jony",lastName:"Ive",
roles:["DESIGN_LEAD","BOARD_MEMBER"],**salary:450000**}
]

Best practices

Access Control Best Practices: Design

- Closed Policy
- Access control policy
- Unpredictable resource ids
- Record ownership enforcement

Access Control Best Practices:

- Enforcement on every request
- Least privilege
- Token invalidation after logout
- Usage of trusted information only
- Access control in backend

Verify tokens

Transmission	Authorization: Bearer [base64(token)]
Authorization	Token Signature
	Issuer (iss) and target audience (aud)
	Username, subject (sub)
	Validity (iat, exp, jti)
	Roles (roles)

Header:

```
{
    "typ": "JWT",
    "alg": "HS256"
}
```

Payload:

```
{   "iss": "https://sso.mydomain.com/"
    "aud": "https://targethost.com/“,
    "sub": "123456789",
    "name": "John Doe",
    "jti": "5e1b0358-b068-4164-9427-00c248b053c0",
    "iat": 1589466636,
    "exp": 1589470236,
    "roles": [
        "admin",
        "superuser"
    ]
}
```

Signature:

Signature(base64(header, payload))

Access Control Best Practices: Testing and Operation

- Extensive testing
- Logging and reporting
- Rate limiting
- No directory listing
- No unnecessary files in web root

Make sure to segregate duties

- Separate control
  - E.g. production vs. developer access
  - But also within an application
- Reduces risk
  - Errors
  - Fraud
- Review and audit regularly

Exercise

Identify the problem

1. What's wrong here?
2. How can you fix the issue?

python Copy code

```python
@app.route('/<role>/dashboard')
def dashboard(role):
    if role == 'admin':
        return render_template('admin_dashboard.html')
    elif role == 'user':
        return render_template('user_dashboard.html')
    else:
        return "Invalid role!"
```

Excursion: OAuth / OIDC

OAuth and OIDC have different purposes

- OAuth is about authorisation
  - Uses tokens
  - Mainly for APIs
  - Enables (partial) delegation/federation of authorisation decision
- OIDC is about authentication
  - Built on top of OAuth
  - Federation protocol
- Example: logging in with social media account

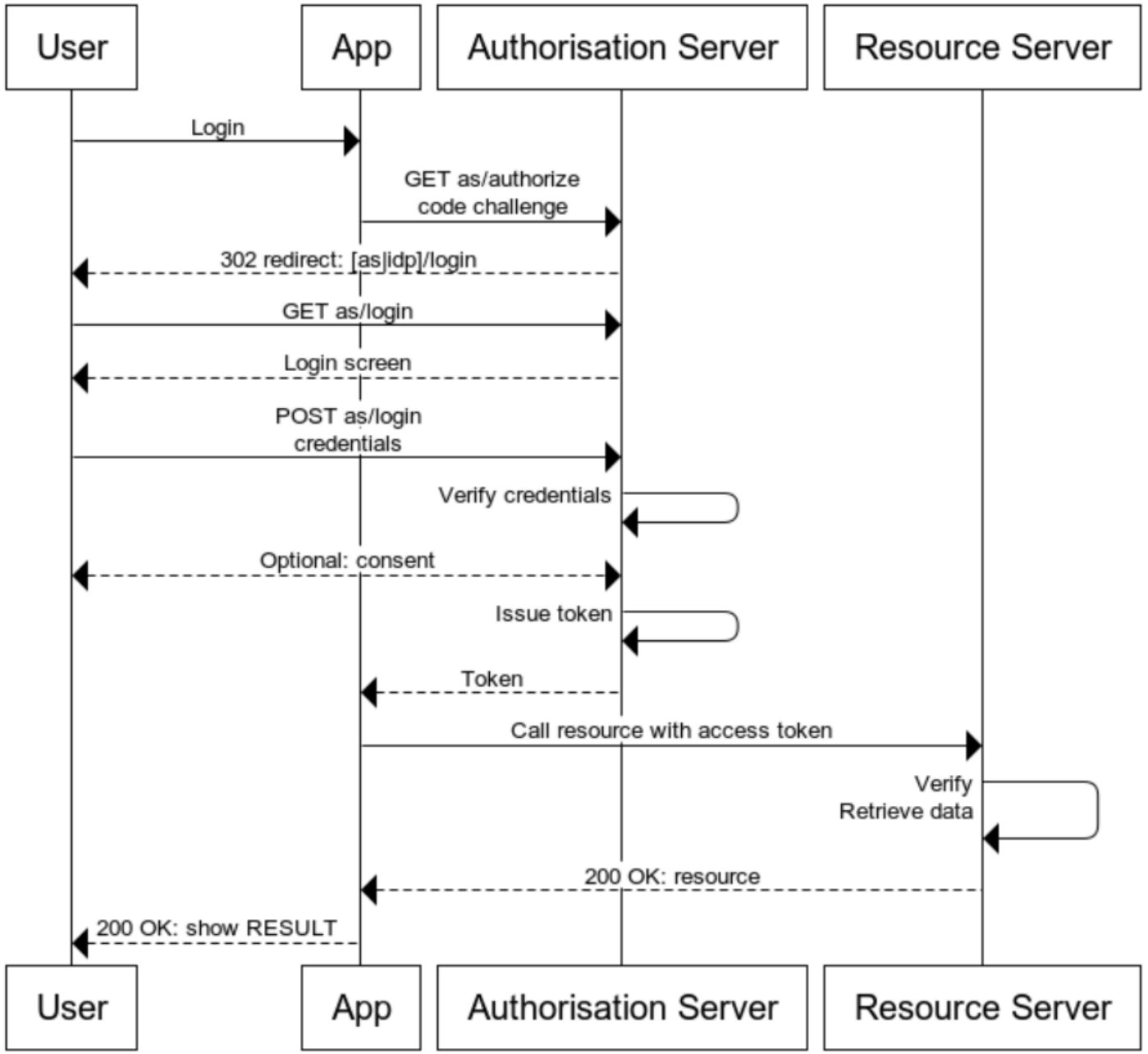
There are different roles in a flow

- Resource Owner
- OAuth/OIDC Client
- Authorisation Server/IdP
- Relying Party

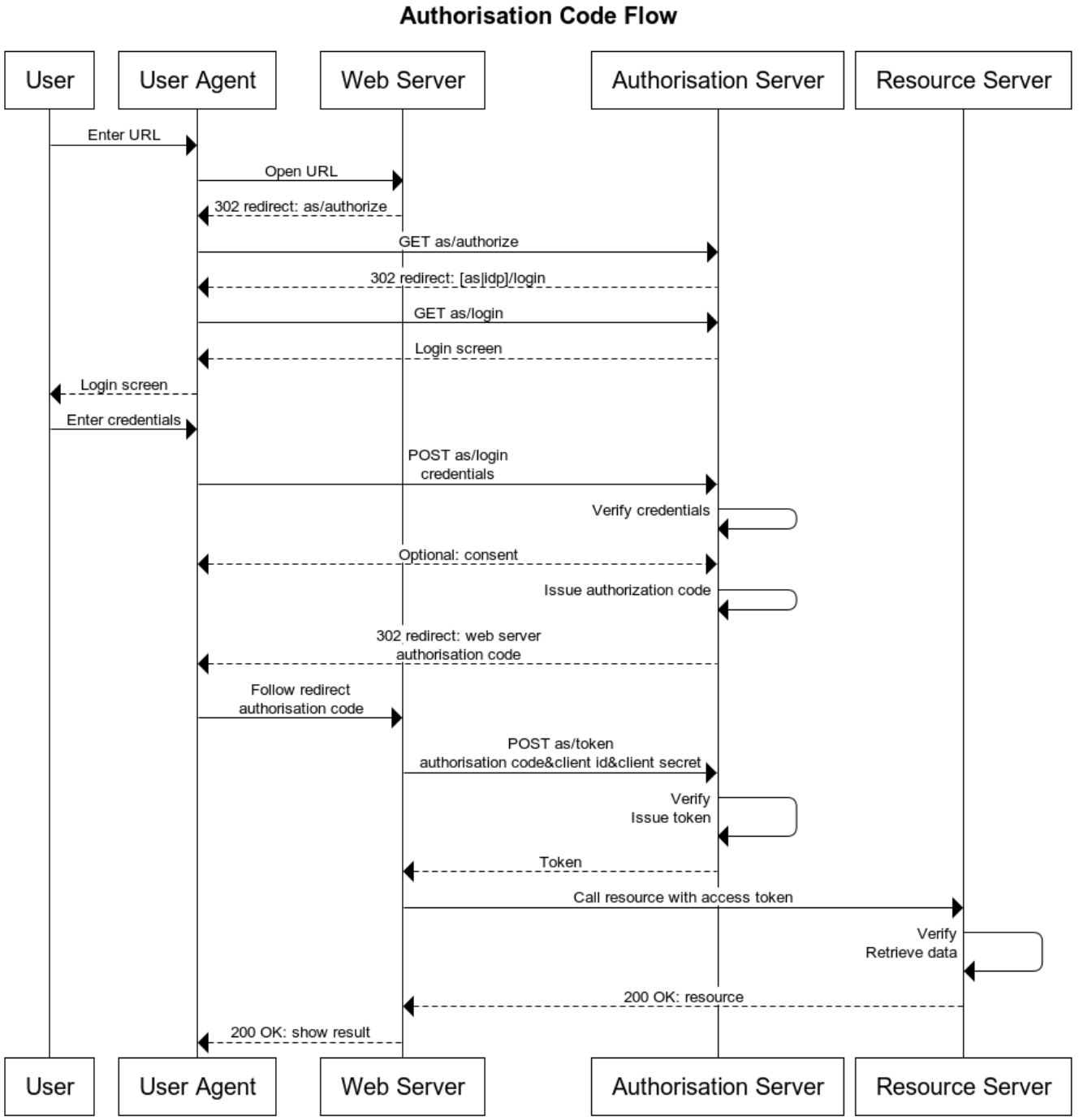
Several flows exist

- Discontinued: Implicit
- Discontinued: Password
- Authorisation Code w/o PKCE
- Device Code
- Others: Refresh, Token Exchange

Implicit Flow

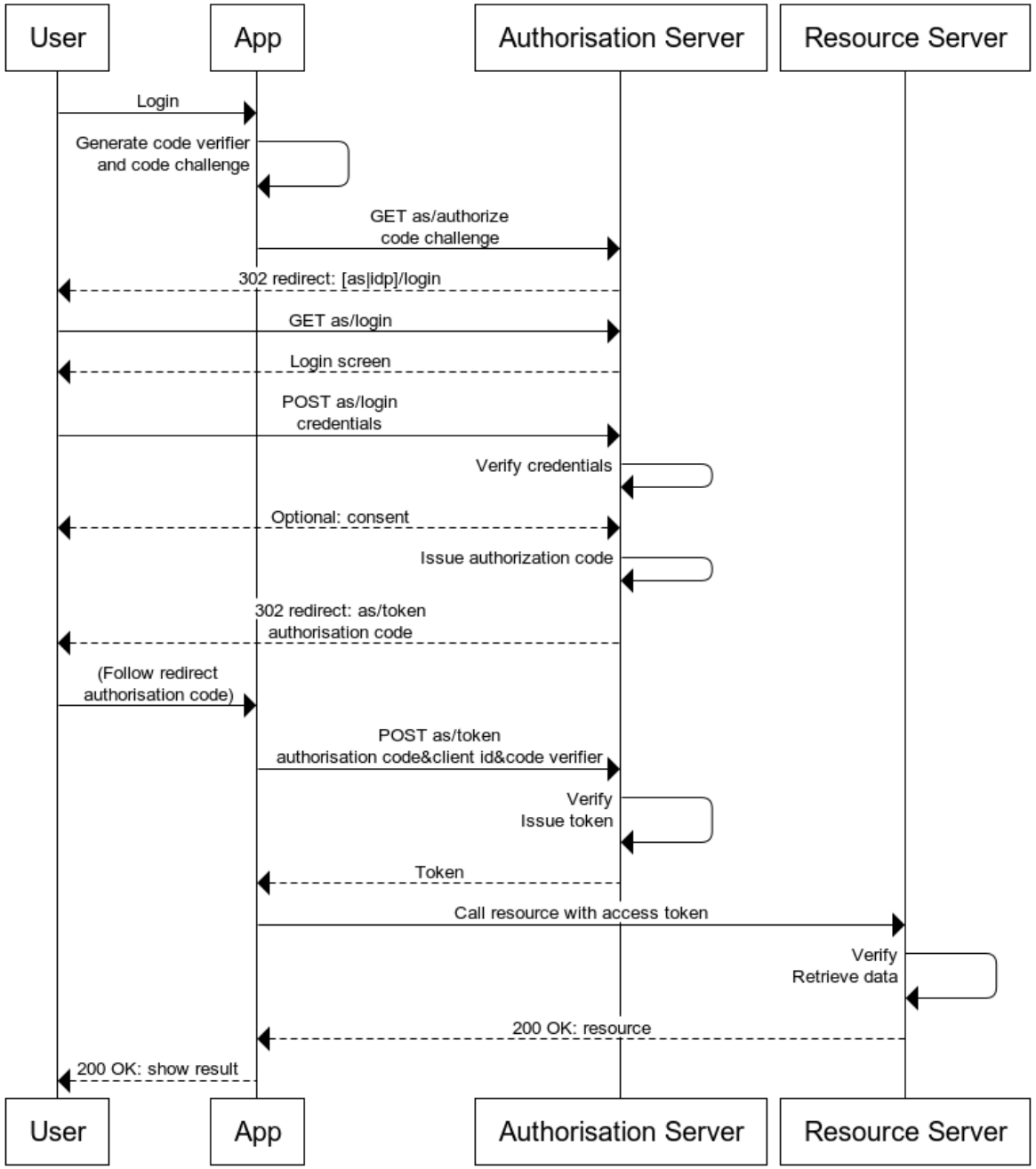


www.websequencediagrams.com



www.websequencediagrams.com

Authorisation Code Flow with PKCE



www.websequencediagrams.com

Excercises

- https://overthewire.org/wargames/natas/